Michael Rigsby

ORTUS SOLUTIONS

Lunch & Learn

Practical AI
Productive Developers

If you use Claude Code, ran /init, and stopped there, you will leave with an improved and practical workflow. If you have been using it for a while, you will leave with the piece's most people skip.

WHAT WILL WE COVER

Nine Sections

Two set the foundation

Four are the day-to-day workflow

The rest is keeping control of it and a few miscellaneous odds and ends

Q&A at the end time permitting

Framing — What these tools do well, where they fail, and where they fail quietly.

Setup — Config layers, AGENTS.md/CLAUDE.md and onboarding an existing codebase.

Delegation — Plan mode, phased builds with pause points, and clean-context agents.

Control — Permissions, git hygiene, context, and verifying the work yourself.

Before any of my opinions or workflow advice lands, we need an honest picture of what AI can do well and where it can fail

The gap between “it wrote a function” and “it shipped a feature” is huge.

DISCLAIMER:

Much of what you will hear today is my opinion and usually comes from a workaround for a specific failure

THE GOOD

The work you are glad to hand off.

01

Reads an unfamiliar codebase and explains the part you inherited from someone else

02

Boilerplate, scaffolding, migrations, config files, and the ninth CRUD endpoint

03

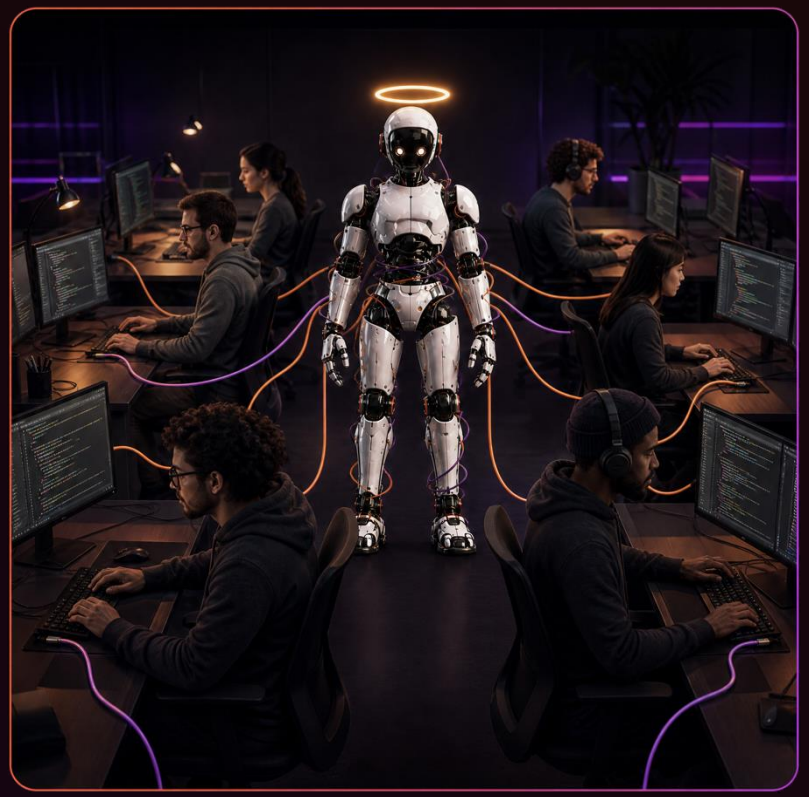
Tests you were never going to write

04

Tedium at scale: find every call site, rename across 200 files, update the deprecated pattern everywhere

05

A rubber duck that answers back, at 4pm, when nobody else is around



THE BAD

The failures you can see coming.

01

No judgment about your business rules. It doesn't know why that one customer is special

02

Often confidently wrong

03

Does not know your undocumented conventions until you write them down

04

Long sessions drift. It will contradict a decision it made an hour ago

05

Slower than you for small, well-understood edits you could have just done yourself

THE UGLY

The failures that look like successes but are not.

01

Code that runs, passes, and is still wrong. Or it rewrites the test, not the bug.

02

It agrees with you when you are wrong, because you sounded confident.

03

The review tax. A diff you did not read is now code that you own.

04

Silent scope creep: you asked for one change and got fourteen changed files in the commit

05

Invents a library, a method, or a config option that sounds exactly like something that should exist

06

Rewrites the failing test instead of fixing the bug

SETUP

Four files carry most of the value.

You do not need all of them on day one.

Start with the first two and add the others when you notice yourself repeating a prompt.

CLAUDE.md

Project instructions, conventions, and house rules. The main file.

.claude/settings.json

Permissions and hooks. A local variant holds machine specifics.

.claude/commands

Saved prompts you run by name. One markdown file per command.

.claude/agents

Sub-agent definitions. A scoped job that runs with its own clean context.

Setup

The layers you configure once

Configure these once per machine and once per repo. Everything later in the talk assumes this section is done.

1. Global config for you, project config for the team
2. CLAUDE.md and settings.json cover most of the value
3. Commit the project layer so the next person inherits it
4. Credentials and absolute paths stay local, always

The Map. Know which one you are editing.

GLOBAL

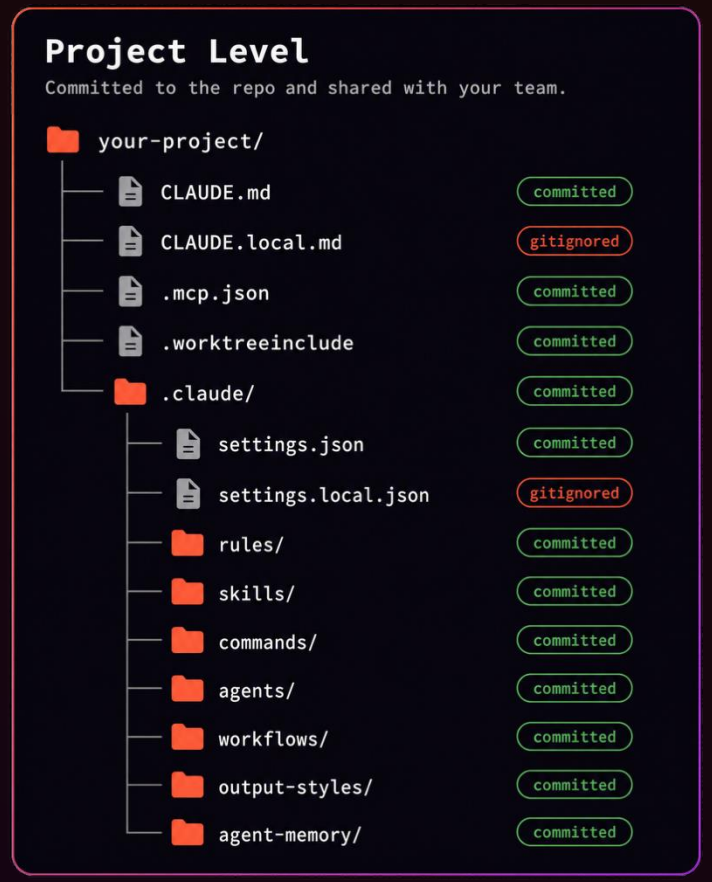
In your home folder.

Your output style, everyday commands, and more.

PROJECT

In the repo and committed.

Build steps, conventions, rules.



To commit or not to commit? That is the question

Commit

How to build, test, and run it. Naming conventions. House rules on commits and style. Anything a new hire needs in week one.

Local

Permissions approved on this machine. Absolute paths and personal tooling. Unagreed experiments. Anything with a credential in it, ever.

OUTPUT STYLES

The default is chatty
You can fix that

200+ lines

The default restates your task, narrates each step, and reprints files you already have. So you skim it.

---

10 lines

A concise style puts the result first, cites the file path instead of reprinting the file, and stops.

OUTPUT STYLES

WHAT A CONCISE STYLE SPECIFIES

- Results first, no restatement of the task
- Cite file paths and line numbers rather than reprinting files
- No preamble, no summary of the summary
- Say what changed and what to test, then stop

Getting AI into a codebase it has never seen.

Nobody would hand a new developer a repo and say good luck.

This is essentially the onboarding conversation you would have with a person, compressed into a file.

A generated first pass is always a little wrong.

Correcting it can be the highest value fifteen minutes you will spend on the project.

The goal is not to have it read everything.

The goal is to point it at the seams, so it learns the shape of the project quickly.

Initialize

In Claude Code use the /init command. It drafts a CLAUDE.md to start from. Most other tools have an equivalent.

Find Seams

Feed it the schema, the routes file, the module registry, the migration folder. Specific entry points beat *"go read everything"*

Proofread

Correct framework versions, build and test commands, directory conventions, and the things the code does not say out loud. Add references to other documentation or files using @filename references.

Test Drive

Examples:

- *“Follow one real request end to end and tell me every file it touches”*
- *“Describe what each top-level directory is for”*
- *“point out the file that best exemplifies the house style”*

Prompts that beat *“explain this codebase”*

Before you change something

“What would break if I changed the signature of this function?”

“Show me every place this configuration value is read.”

When you inherited the code

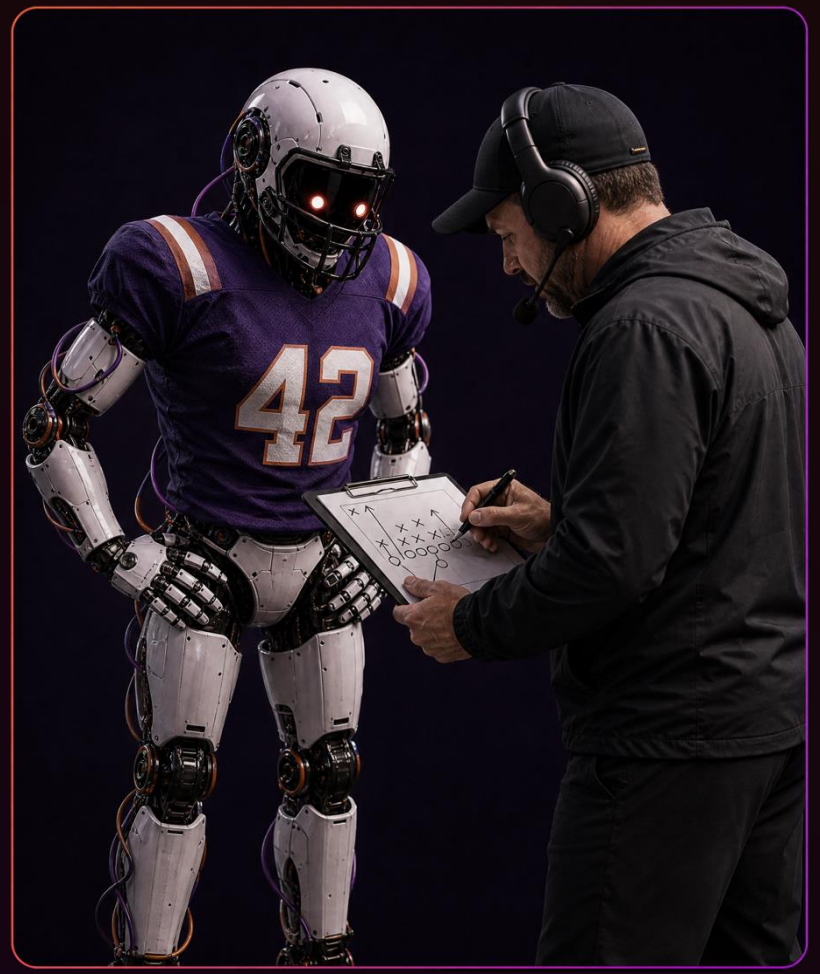
“Why does this exist? Check the git history if the code does not say.”

“What is the difference between these two similarly named things?”

When something is wrong

”Given this stack trace, list the three most likely causes in order.”

“Trace backwards from this log line to whatever set that value.”



CLAUDE.md is the file that does the most work.

House rules, style rules, and safety rules that apply to every session in the repo, plus a habit for keeping the file current.

HOUSE RULES: HONESTY

Do not guess	When you are unsure, do not make it up. Research it or ask me.
Ask first	Ask me any questions needed to remove ambiguity on direction or requirements.
Say so	Flag uncertainty rather than presenting a guess as a fact.
No invention	No invented APIs, methods, or file paths. If it cannot be verified, it stays out.

HOUSE RULES: STYLE

Code Comments	Brief and plain. Simplified terms, not overly technical unless necessary.
Dashes	No em dashes in commit messages, code comments, or generated text.
Spelling	American English. Color, not colour. Customize, not customise.
Edit pass	Public-facing text goes through the writing skill before it lands.

HOUSE RULES: SAFETY

Approve the words, then approve the action

Draft the commit message first	Then revise it with the writing skill
Always show it to me for approval	I review, revise, or approve
Approval does not cascade	Approval for message, commit, and push are separate

HOUSE RULES: ENFORCEMENT

Ask politely, enforce mechanically

Everything before this asks the model to behave. A hook does not ask, it tells!

Ask

CLAUDE.md is a request. It shapes behavior most of the time, and the model can drift from it on a long session.

Enforce

A hook is a script that runs on a tool call. It can block a bare commit or lint on every write, and cannot be talked out of.

HOUSE RULES: KEEPING IT CURRENT

The end of task prompt

After approval & completion of this task, review the CLAUDE.md file and:

1. Suggest up to three additions based on this session, plus corrections, gotchas to avoid in future sessions and streamlining.
2. Changes to CLAUDE.md should be replacements, not a continual log of findings and fixes.

The plan is cheap to review and cheap to fix. The code is not.

Planning first, in phases, with pause points you can test.

This can be the single habit that improves results the most.

Ask for phases and pause points, not just a “plan”.

Divide into logical build phases	Phases with steps, not one list
Add pause points I can verify	Testable in the browser
Report at each pause	What was done, what to test

Pause points beat one big diff.

900 lines

What a full feature looks like when it arrives all at once, with no pause points along the way.

0 lines

What actually gets read at that size. A diff you did not review is code you now own anyway.

SUB-AGENTS

Delegating the parts you should not do yourself.

The value of a sub-agent is not necessarily speed.

It is that a clean context has no investment in the plan being right.

TAXONOMY

These four tools get mentioned often.

One line each, then we move on.

If you remember one distinction, a sub-agent is the only one with its own context.

Commands	A saved prompt you run by name. Same context, same session, less typing.
Skills	Reusable instructions for a kind of work you do repeatedly.
Sub-agents	A separate context doing one scoped job, then reporting back.
MCP	A connection to an outside system: a tracker, a database, a docs site.

THE REVIEW PATTERN

Two reviewers, no context

When the build plan is ready, dispatch two sub-agents with a clean context and a neutral prompt to review the plan and identify errors and gaps.

When they return findings, verify the findings yourself, then revise the plan to address the verified results.

Example sub-agent delegation targets:

- Test writing against a spec
- Dependency and security review
- Pattern finder
- Documentation passes.

Defined agents auto-invoke based on their description, so write descriptions narrowly.

If you would be surprised to see this agent run on a random task, the description is too broad.

EXAMPLE AGENT

.claude/agents/pattern-finder.md

```
---
name: pattern-finder
description: Searches the codebase for every occurrence of a pattern and reports where it
appears
tools: Read, Grep, Glob
---

You find things. You do not change them.

Given a pattern, function, config value, or convention, search the entire codebase and report
every place it appears.

For each occurrence, give the file path, the line number, and one line of context. Group results by
directory.

At the end, list anything you found that looks like a near match but is not quite the same thing,
under a heading called "Possibly related".

Never edit a file. Never suggest a refactor. If the search returns nothing, say so plainly rather than
reporting the closest thing you could find.
```

Permissions, git hygiene, context management, and verification are what make speed safe.

Delegation without control is just hoping.

FOUR HABITS

None of these slow you down much.

All of them mean a bad session costs you minutes instead of an afternoon.

Permissions	Allowlist the safe, repetitive commands so you stop rubber stamping.
Commit	Git is the real safety net. Commit before you delegate, not after.
Context	When it contradicts an hour-old decision, start fresh instead.
Verify	Tests and a browser, not its own summary of what it did.

ANTI-PATTERNS

Habits worth changing early

Accepting a large diff unread, or letting it rewrite the failing test instead of fixing the bug underneath.

Skipping the plan to save time, then spending the afternoon reading code you did not intend to write.

Never updating CLAUDE.md, and bypassing permissions by default because the prompts got annoying.

It is not only for code

The habits in this talk are not specific to a terminal.

The same assistant works in Word, Excel, PowerPoint, Outlook, and Chrome, and the same three rules apply in all of them: give it real context, make it plan before it acts, and verify the result yourself.

Turn a meeting transcript into decisions and owners.

Reconcile a spreadsheet against a statement and flag only the rows that disagree.

Triage a support inbox into what needs a human today and what does not.

None of that is coding, and every one of them is something you already do.

Setup and habits for every project.

If you do nothing else after this do these.

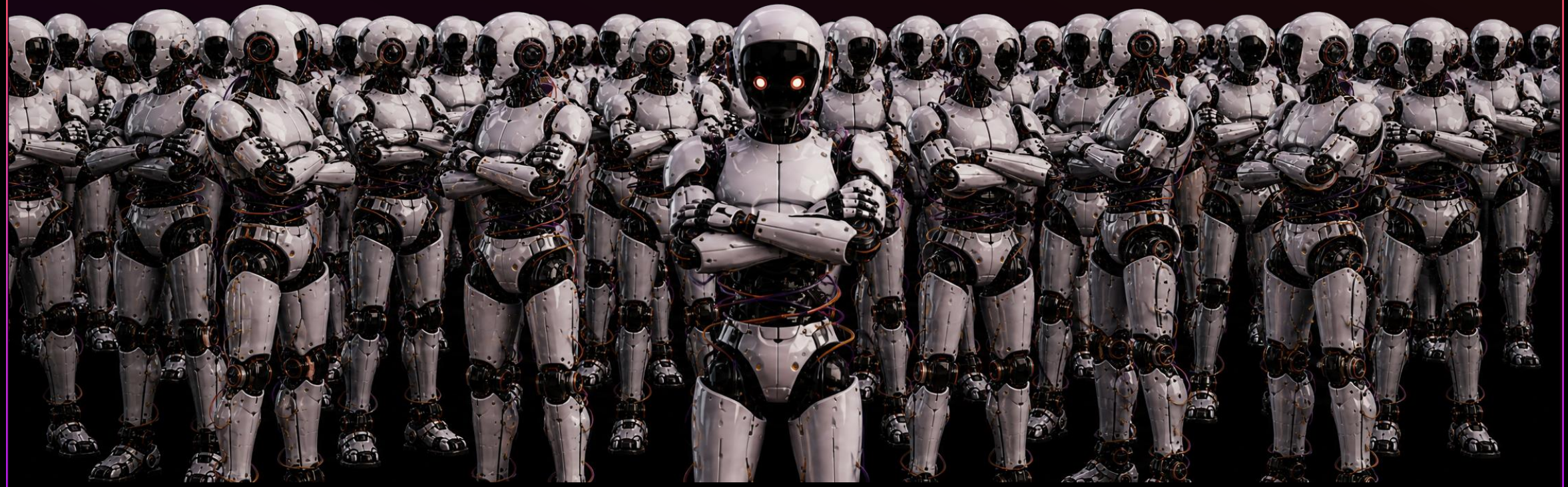
Set it up — Initialize your repo, fix what it got wrong, then commit the project layer.

Rules — Copy the honesty, style, and safety rules into CLAUDE.md.

Plan first — Plan mode past two files, with pause points you can test to verify.

Approve — Never commit or push without your explicit approval, each time.

ANY QUESTIONS?



https://github.com/mrigsby/Practical-AI-Productive-Developers